COLLEGE OF ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING

FUNDAMENTALS OF PROGRAMING II: SWEG2102

TITLE: LIBRARY MANAGEMENT SYSTEM

SECTION B

No	**Group members name**	**Id. No**
1	Eyob Tamiru	ETS 0549/17
2	Ermiyas Alemayehu	ETS 0511/17
3	Eyob Arega	ETS 0538/17
4	Ermiyas Getachew	ETS 0512/17
5	Eyerusalem Hailu	ETS 0534/17

Submitted to: Mrs. Eleni Teshome

Submission date: Apr 15, 2026

Contents

LIBRARY MANAGEMENT SYSTEM 1
Introduction 1
Problem Statement 1
Proposed Solution 2
Objectives 2
System Architecture 3
1. Presentation Layer (User Interface – C++) 3
2. Application Layer (Business Logic – C++) 3
3. Data Layer 3
Functional Requirements 4
Non-Functional Requirements 4
Database Design 5
Expected Outcomes 5
Conclusion 6

LIBRARY MANAGEMENT SYSTEM

Introduction

A Library Management System (LMS) is a simple software application used to organize and manage library activities such as storing book records, issuing and returning books, and keeping track of users.

Manual library systems are often slow and difficult to manage. They can lead to mistakes, lost records, and wasted time, making the overall process inefficient.

This project proposes the development of an advanced Library Management System using C++, integrated with:

- MySQL Database for persistent and structured data storage
- File Handling System for backups, logs, and reports
- Authentication System for secure access control
- Search System for fast and efficient data retrieval

The system combines core programming concepts with backend design principles to deliver a scalable and reliable solution.

Problem Statement

Many libraries, particularly in academic or small institutional settings, still rely on manual methods (registers, index cards, and basic spreadsheets) for managing their inventory and tracking borrowed items. This manual system leads to several critical issues:

- **Inefficient Search & Retrieval**: Finding the exact location or availability status of a book is slow, leading to frustrated patrons and staff.
- **High Error Rate in Record-Keeping:** Manual data entry frequently results in errors regarding ISBNs, issue/return dates, and patron details, complicating fine calculation and accountability.
- **Lack of Real-Time Inventory Control**: Staff cannot instantly determine the total number of copies available for a specific title, leading to inconsistent borrowing policies.

Proposed Solution

The proposed system enhances the traditional LMS by integrating:

- ✓ Database System (MySQL)
  - ➢ Stores books, users, and borrowing records
  - ➢ Ensures data persistence and integrity
  - ➢ Supports complex queries and relationships
- ✓ File Handling System
  - ➢ Backup of database records into files
  - ➢ Logging of transactions (borrow/return)
  - ➢ Export reports (e.g., overdue books)
- ✓ Authentication System
  - ➢ Secure login system
  - ➢ Role-based access (Admin / User)
  - ➢ Prevent unauthorized access
- ✓ Search System
  - ➢ Search books by title, ID, or author
  - ➢ Fast retrieval using SQL queries

Objectives

4.1 General Objective

To design and implement a secure, efficient, and scalable Library Management System using C++ integrated with database and file systems.

4.2 Specific Objectives

- ➢ Implement CRUD operations using MySQL
- ➢ Design authentication with role-based access
- ➢ Develop file-based backup and logging system
- ➢ Enable efficient search functionality
- ➢ Ensure data consistency and integrity
- ➢ Apply modular programming and error handling

System Architecture

The system follows a **layered architecture**, separating concerns into presentation, logic, and data layers for better organization and maintainability.

1. Presentation Layer (User Interface – C++)

- Handles all user interactions, Provides menus for operations such as:
  - Add / search / update / delete books
  - Issue and return books
  - View reports (overdue, available books, etc.)
- Takes input from users and displays output results.

2. Application Layer (Business Logic – C++)

- Acts as the core processing unit of the system, Validates user inputs before processing, Implements system rules such as:
  - Checking book availability before issuing
  - Calculating due dates and fines
  - Preventing duplicate records
- Communicates with both the database and file system.

3. Data Layer

This layer is divided into two components:

a) MySQL Database (Primary Storage)

- Stores all structured and critical data:
  - Books (ISBN, title, author, copies)
  - Users (ID, name, contact)
  - Transactions (issue/return records, due dates)
- Supports CRUD operations for efficient data management.

b) File System (Backup & Logs)

- Stores:
  - Backup copies of important data
  - System logs (transactions, errors, activities)
- Used for recovery and auditing purposes.

Functional Requirements

6.1 User Authentication

- ✓ Login with username and password
- ✓ Role-based access (Admin / User)

6.2 Book Management

- ✓ Add, update, delete books (Admin only)
- ✓ Track availability status

6.3 Member Management

- ✓ Register and manage users

6.4 Borrowing System

- ✓ Issue and return books
- ✓ Track due dates and overdue records

6.5 Search System

Search by:

- ➢ Book ID
- ➢ Title

6.6 File Handling Features

- ➢ Backup database to file
- ➢ Restore from backup
- ➢ Log transactions
- ➢ Generate reports

Non-Functional Requirements

- ➢ Security: Use authentication and prepared statements
- ➢ Reliability: Use transactions for critical operations
- ➢ Scalability: Database supports large data
- ➢ Usability: Simple text-based interface
- ➢ Maintainability: Modular code structure

Database Design

Tables:

Users

- id
- username
- password
- role

Books

- id
- title
- author
- available

Borrow_records

- id
- book_id
- user_id
- borrow_date
- return_date

Expected Outcomes

The system is expected to automate all core library operations, reducing reliance on manual processes and improving overall efficiency. It will provide secure user access through controlled authentication, ensuring that only authorized users can perform specific actions within the system.

Additionally, the system will ensure data persistence and integrity by storing information reliably and minimizing inconsistencies or data loss. It will support fast and efficient searching capabilities, allowing users to quickly locate books and relevant records.

Furthermore, the system will maintain detailed logs and backups to support monitoring, auditing, and recovery processes. Overall, these features will significantly reduce human errors and enhance the accuracy and reliability of library management.

Conclusion

This enhanced Library Management System goes beyond basic academic requirements by incorporating practical components such as database integration, user authentication, and file system management.

It also serves as a solid foundation for understanding core concepts in backend development, system design, and data management using C++, effectively bridging the gap between theoretical knowledge and practical implementation.